QUERY HTTP method

The **HTTP QUERY method** is **a newly standardized HTTP request verb defined in RFC 10008 in June 2026**. It acts as a specialized hybrid designed specifically for **rich, complex data searches**. It bridges the operational gap between `GET` and `POST` by allowing a **structured request body while remaining strictly safe and idempotent**.

The Problem it Solves

Historically, developers building complex search APIs faced an architectural dilemma:

- **Using `GET`**: Complex filters, pagination, and multi-layered conditions forced parameters directly into the URL query string. This frequently hit character length limits (typically around 8 KB), raised data privacy issues in logs, and lacked an elegant way to encode complex objects or nested arrays.
- **Using `POST`**: To bypass URL limits, developers passed parameters inside a JSON or URL-encoded request body using `POST`. However, `POST` is semantically defined as non-idempotent (not safe for automatic retries) and its responses are not natively cached for data fetching.

Key Characteristics

The HTTP QUERY Method introduces the best properties of both older methods:

- **Has a Request Body**: Similar to `POST`, its query details are sent cleanly inside the payload, liberating developers from URL character caps.

Ask anything

Core Properties Comparison

This table shows how `QUERY` compares to its sibling methods:

Property	`GET`	`QUERY`	`POST`
Safe & Idempotent	Yes	Yes	No
Contains Request Body	No (undefined semantics)	Yes (Expected)	Yes (Expected)
Primary Intent	Basic, simple retrieval	Complex data search/filtering	Data creation or execution
Cache Behavior	Native and ubiquitous	Fully supported (including body)	Restricted / Non-standard for reads

Example Wire Format

A raw `QUERY` call looks structurally identical to a `POST` request, but utilizes the `QUERY` verb:

http

```
QUERY /api/v1/analytics/search HTTP/1.1
Host: example.org
Content-Type: application/json

{
  "filters": {
    "status": "active",
    "categories": ["electronics", "home"],
    "price_range": { "min": 100, "max": 500 }
  },
  "sort": "-created_at",
  "limit": 50
}
```

Use code with caution.

If you are thinking about implementing this, let me know **what language or framework** you are using so I can guide you through the setup.

How exactly will caching work with QUERY? who is responsible for recognizing that two json bodies are equivalent or not?

Under RFC 10008, caching with the `QUERY` method works by **including the content of the request body directly into the cache key calculation**. This marks a major shift from traditional `GET` caching, which only uses the request URL.

The **cache intermediary** (such as a CDN, reverse proxy, or browser cache) is responsible for generating this key and determining payload equivalence. However, network intermediaries **do not parse JSON to check for semantic equivalence**. Instead, they rely strictly on a byte-for-byte comparison of the raw payload string.

How Caching Intermediaries Process a QUERY

When an intermediary receives a `QUERY` request, it builds the cache key using a specific sequence:

1. **Key Generation**: The cache constructs a signature, usually a cryptographic hash like `Hash(URL + Request_Body)`.
2. **Byte-for-Byte Matching**: Intermediaries treat JSON as an opaque string.
3. **Whitespace Sensitivity**: If two requests are logically identical but contain different formatting, spacing, or key ordering, they will generate different hashes, resulting in a cache miss.

Who Handles JSON Equivalence?

Because network intermediaries operate at the raw HTTP layer, responsibility for matching complex JSON strings is split across three areas:

```
[ Client App ]  ──► Normalizes JSON structure (keys, whitespace)
       |
[ Intermediary/CDN ]  ──► Performs strict, fast byte-level hash checks
       |
[ Origin Server ]  ──► Handles semantic validation if the cache misses
```

- **The Client Application (Primary Responsibility)**: The client must normalize the payload before sending it to maximize cache hit rates. This requires sorting JSON object keys alphabetically and removing unnecessary whitespace (`JSON.stringify()` native ordering is not guaranteed).
- **The Intermediary/CDN (Strict Byte Matching)**: Intermediaries like Cloudflare, Fastly, or Nginx do not unmarshal JSON, as doing so would destroy edge routing performance. They strictly match exact string structures.
- **The Origin Server (Semantic Matching)**: If a request misses the edge cache because a client sent un-normalized JSON, the origin server parses the request body, processes the database query, and returns a standard `ETag` validator header in the response.

Server Optimization with Content-Location

To prevent cache fragmentation from minor JSON variations at the network edge, the `QUERY` specification allows origin servers to return a `Content-Location` header:

http

```
HTTP/1.1 200 OK
Content-Location: /api/v1/analytics/search?id=hash_xyz
Cache-Control: max-age=3600
```

Use code with caution.

This maps the complex query response to a standard, unique query URL. Subsequent clients can then perform a regular `GET` request directly to that specific URL, bypassing the body-hashing requirements completely.

If you are setting up caching rules, let me know **which CDN or proxy** you are deploying to so we can structure your edge cache keys correctly.

12 sites

New HTTP QUERY Method : r/programming - Reddit
The response to a QUERY method is cacheable; a cache MAY use it to satisfy...
Reddit

RFC 10008: The HTTP QUERY Method
* 1. Introduction. This specification defines the HTTP QUERY request method as a...
RFC Editor

The new HTTP QUERY method explained | Kreya
The new HTTP QUERY method explained * Queries using GET Traditionally, if you...
Kreya

Show all